Chapter 12 ~ Working with Cookies and Sessions

Preface: Web applications keep track of users as they move through an application. In this section you are exposed to cookies and sessions which are the PHP tools utilized for keeping track of users. Without the server tracking users, there can be no shopping cars or custom website personalization. As you will discover, the difference between cookies and sessions is that cookies store data in the user's browser, whereas sessions store data on the server. Sessions are more secure than cookies and have the capability of storing more data.

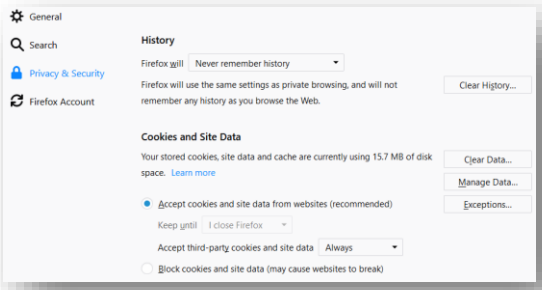
How about some Cookies?

Cookies provide a way for a web application to store information in the user's browser and then retrieve it every time the user requests a page. Cookies are abused, for example, some advertisers use cookies to track the websites you have visited. These types of cookies are known as "third-party" because they are sent by the advertisers, not from the visited websites. Fortunately, browsers allow you to block these third-party cookies with certain settings.

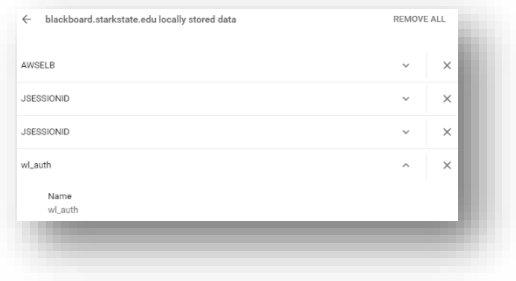
If you aren't familiar with viewing cookies on your system, take a few moments now to do so.

In your "Privacy & Security" section, there will be a category called Cookies and Site Date, (I am referring to Firefox; however, Google is very user friendly as well)

Click on the "Manage Data…" button to explore the stored cookies.



The image below is a snippet of a Blackboard cookie on my Google browser.



How Cookies Work

- A cookie is a name/value pair that is stored in a browser
- It's basically a small text file that lets you store a small amount of data (nearly 4kb)on the user's computer
- On the server, a web application creates a cookie and send it to the browser
- On the client, the browser saves the cookie and sends it back to the server every time it accesses a page from that server
- By default, cookies only last until the user closes the browser; however, cookies can be set to "persists" in the user's browser until a specified expiration date
- Some users disable cookies in the browser settings

Typical Uses for Cookies

- To allow users to skip login and registration forms that gather data like username, password, address, or credit card data
- To customize pages that display information like weather reports, sports scores, and stock quotes
- To focus advertising like banner ads that target the user's interests

Set and Get a Cookie

- The setcookie() function is used to set a cookie
- This function must be called before any output generated by the script otherwise the cookie won't be set
- The basic syntax of the function is as follows:

```
setcookie(name, value, expire, path, domain, secure);
```

Cookie Parameter	Description
$name	The name of the cookie
$value	The value of the cookie, default is an empty string
$expire	The expiration date as timestamp; if set to 0 will expire upon browser exit; default is 0
$path	Specify the path on the server for which the cookie will be available. If set to /, the cookie will be available within the entire domain
$domain	Specify the domain for which the cookie is available for example: www.example.com
$secure	This field, if present, indicates that the cookie should be sent only if a secure HTTPS connection exists
$httponly	If TRUE, the cookie is only made available through the HTTP protocol and not through client-side scripting like JavaScript

setCoolieEx.php

```php
<?php
// Setting a cookie
setcookie("username", "Mickey Mouse", time()+30*24*60*60);
?>
```

Working with Sessions

A session is way to store information in variables to be used across multiple pages. Unlike cookies, the information is NOT stored on the user's computer.

What is a Session?

A PHP session stores data on the server rather than user's computer. In a session based environment, every user is identified through a unique number called session identifier or SID. This unique session ID is used to link each user with their own information on the server like emails, posts, etc.

The session IDs are randomly generated by the PHP engine which is almost impossible to guess. Furthermore, because the session data is stored on the server, it doesn't have to be sent with every browser request.

Starting a Session

```php
<?php
// Start the session
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Set session variables
$_SESSION["favcolor"] = "green";
$_SESSION["favanimal"] = "cat";
echo "Session variables are set.";
?>

</body>
</html>
```

Get PHP Session Variable Values

Create another page that will access the session information set in the first page.

```php
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Echo session variables that were set on previous page
echo "Favorite color is " . $_SESSION["favcolor"] . ".<br>";
echo "Favorite animal is " . $_SESSION["favanimal"] . ".";
?>

</body>
</html>
```

Another way to display all of the session variables.

```php
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
print_r($_SESSION);
?>

</body>
</html>
```

Delete Sessions

```php
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// remove all session variables
session_unset();

// destroy the session
session_destroy();
?>

</body>
</html>
```

Working with a Shopping Cart

The best way to understand Cookies and Sessions is to actually work with an application where they are incorporated.

I have provided you with the complete Shopping Cart code for you to review along with my notes.

This shopping cart consists of two (2) pages for the user interface:

1. Add Item
    a. Allows the user to add an item to the cart
    b. Also includes a link so the user can view the cart without adding an item
2. Cart
    a. Displays all of the items in the user's cart along with a subtotal
    b. From this page, the user can update the quantities for the items in the cart by changing the quantity and clicking the Update Cart button.
    c. The user is able to remove an item from the cart

The index.php page uses the functions in the cart.php file to add and update items into the cart. It also uses the add_item_view.php and cart_view.php files to display the Add Item page and the Cart page.

The point of this application is to illustrate the use of cookies and sessions, the user interface is not realistic. Hopefully, you understand why it isn't realistic, the data isn't stored in a database or store the session in a database.

The index.php file

Begins by starting a session, the first statements establish the session along with the timeframe, which in this example is two (2) weeks. Notice other timeframes are commented out. If the user closes the browser and returns within two weeks, the session is continued.

index.php

```php
<?php
// Start session management with a persistent cookie
$lifetime = 60 * 60 * 24 * 14;          // 2 weeks in seconds
// $lifetime = 0;                          // session cookie
//$lifetime = 60 * 60 * 24 * 365 * 4;     // 3 years in seconds

session_set_cookie_params($lifetime, '/');
session_start();

```

The next line of code checks whether the cart array is empty in the $_SESSION array, if so, an array is created.

```
// Create a cart array if needed
if (empty($_SESSION['cart'])) { $_SESSION['cart'] = array(); }

```

Once the cart is initialized, a multi-dimensional array of products is created. In a real-world environment, this would naturally be a database.

```
// Create a table of products
$products = array();
$products['FURN-1754'] = array('name' => 'Bakers Cabinet', 'cost' => '249.50');
$products['FURN-6289'] = array('name' => 'Rocker', 'cost' => '175.50');
$products['FURN-3408'] = array('name' => 'Hutch', 'cost' => '399.50');

```

Now it's time to load the cart.php file which contains the functions for working with the cart array. The functions have the capability of adding a new item to the cart or updating the quantity of an existing item.

```

// Include cart functions
require_once('cart.php');

```

Next the action has to be performed based on the action parameter of the POST or GET request. If the action parameter hasn't been set, the code uses a default action of "show_add_item".

```

// Get the action to perform
$action = filter_input(INPUT_POST, 'action');
if ($action === NULL)
{
    $action = filter_input(INPUT_GET, 'action');
    if ($action === NULL)
    {
        $action = 'show_add_item';
    }
}

```

Once the action has been established, a switch statement is used to perform the action. If the action is “add”, the code retrieves the new product key and quantity from the POST request and passes it to the add_item() function, which adds the item to the cart and displays the Cart page.

If the action is “Update”, the code retrieves the array of new quantities from the POST request, it then uses a loop to check each quantity in the array. If the new quantity is different than the old quantity, the code call the update_item() function and then displays the Cart page.

If the action is “show_cart”, the code displays the Cart page. This occurs when the user clicks on the View Cart link on the Add Item page.

If the action is show_add_item, the code displays the Add Item page. This occurs when the user clicks on the Add Item link on the Cart page.

If the action is empty_cart, the code uses the unset() function to remove the cart from the $_SESSION variable. This occurs when the user clicks the Empty Cart link on the Cart page. The session isn’t ended programmatically; however, from the user’s point of view, the session has ended.

```
32
33  // Add or update cart as needed
34  switch($action)
35  {
36      case 'add':
37          $product_key = filter_input(INPUT_POST, 'productkey');
38          $item_qty = filter_input(INPUT_POST, 'itemqty');
39          add_item($product_key, $item_qty);
40          include('cart_view.php');
41          break;
42      case 'update':
43          $new_qty_list = filter_input(INPUT_POST, 'newqty', FILTER_DEFAULT, FILTER_REQUIRE_ARRAY);
44          foreach($new_qty_list as $key => $qty)
45          {
46              if ($_SESSION['cart'][$key]['qty'] != $qty)
47              {
48                  update_item($key, $qty);
49              }
50          }
51          include('cart_view.php');
52          break;
53      case 'show_cart':
54          include('cart_view.php');
55          break;
56      case 'show_add_item':
57          include('add_item_view.php');
58          break;
59      case 'empty_cart':
60          unset($_SESSION['cart']);
61          include('cart_view.php');
62          break;
63      case 'end_session':
64          // Clear session data from memory
65          $_SESSION = array();
66
```

The rest of the code basically cleans up the session and removes the session ID.

```
        // Clean up session ID
        session_destroy();

        // Delete the cookie for the session
        $name = session_name();                // Get name of the session cookie
        $expire = strtotime('-1 year');        // Create expiration date in the past
        $params = session_get_cookie_params(); // Get session params
        $path = $params['path'];
        $domain = $params['domain'];
        $secure = $params['secure'];
        $httponly = $params['httponly'];
        setcookie($name, '', $expire, $path, $domain, $secure, $httponly);

        include('cart_view.php');
        break;
}
?>
```

The cart.php file

This code modifies the behavior of a shopping cart. It defines three (3) functions, add, update, and sub-total items in the cart.

The add_item() function takes an item key and quantity as its parameters and uses them to add the item to the cart. The code has access to the global “products” array and then checks if the quantity is less than 1. If so, the code exits the function

If the item is already in the cart, the add_item() function adds the existing quantity to the new quantity and calls the update_item() function to update the quantity for the item. It then exits the function.

If the item isn’t already in the cart, the additem() function gets the cost of the item from the products array and calculates the total for the item. Then it creates an array that contains the item’s name, cost, quantity, and total. It then stores the item array in the car array using the item’s key as the index

```php
<?php
// Add an item to the cart
function add_item($key, $quantity)
{
    global $products;
    if ($quantity < 1) return;

    // If item already exists in cart, update quantity
    if (isset($_SESSION['cart'][$key]))
    {
        $quantity += $_SESSION['cart'][$key]['qty'];
        update_item($key, $quantity);
        return;
    }

    // Add item
    $cost = $products[$key]['cost'];
    $total = $cost * $quantity;
    $item = array(
        'name' => $products[$key]['name'],
        'cost' => $cost,
        'qty'  => $quantity,
        'total' => $total
    );
    $_SESSION['cart'][$key] = $item;
}

```

The update_item() function also takes an item key and quantity as its parameters and uses them to update the items quantity and total. The code checks if the specified item is in the cart, if not, the update_item() function doesn't execute any code. If the item is in the cart, the function checks whether the quantity is less than or equal to zero (0). If so, the code uses the unset function to remove the item from the cart. Otherwise, the code stores the new quantity in the car array. It also calculates a new item total and stores it in the cart array.

```
27
28  // Update an item in the cart
29  function update_item($key, $quantity)
30  {
31      $quantity = (int) $quantity;
32      if (isset($_SESSION['cart'][$key]))
33      {
34          if ($quantity <= 0)
35          {
36              unset($_SESSION['cart'][$key]);
37          } else {
38              $_SESSION['cart'][$key]['qty'] = $quantity;
39              $total = $_SESSION['cart'][$key]['cost'] *
40                       $_SESSION['cart'][$key]['qty'];
41              $_SESSION['cart'][$key]['total'] = $total;
42          }
43      }
44  }
```

The get_subtotal() function adds the totals for each item in the cart and returns it as a formatted number. It starts by setting the subtotal to zero (0), then it loops over each item in the cart and adds its total to the subtotal. Finally, it returns the subtotal as a number with two digits after the decimal.

```
45
46  // Get cart subtotal
47  function get_subtotal ()
48      {
49      $subtotal = 0;
50      foreach ($_SESSION['cart'] as $item)
51      {
52          $subtotal += $item['total'];
53      }
54      $subtotal_f = number_format($subtotal, 2);
55      return $subtotal_f;
56  }
57  ?>
58
```

The add_item_view.php file

This page displays a form that lets the user add an item to the cart by selecting a product and a quantity for that item. The page also displays a link that lets the user view the cart without adding an item.

The form uses the POST method to submit the data back to the index.php file for processing. This form includes a hidden field with a name of “action” and a value of “add” to indicate that the index.php file should add the item to the cart.

```
        <form action="." method="post">
            <input type="hidden" name="action" value="add">

```

The first <select> tag has a name of “productkey” which lets the user select a product from a drop-down list. The PHP code uses a foreach loop to generate the <option> tags for the drop-down list. It loops through each item in the products array. The loop formats the cost of each item as a number with two digits, and it uses the item name and formatted cost to generate the text to display for the item.

```
        <label>Name:</label>
        <select name="productkey">
        <?php foreach($products as $key => $product) :
            $cost = number_format($product['cost'], 2);
            $name = $product['name'];
            $item = $name . ' ($' . $cost . ')';
        ?>
            <option value="<?php echo $key; ?>">
                <?php echo $item; ?>
            </option>
        <?php endforeach; ?>
        </select><br>
```

The second <select> tag has a name of “itemqty”. This tag lets a user select the quantity from a drop-down list. It uses a for loop to generate the <option> tags for the drop-down list. The tags display values from 1 to 10. The code uses the index of the loop as both the value that’s submitted to the index.php file and the text that is displayed in the drop-down list.

```
29
30              <label>Quantity:</label>
31              <select name="itemqty">
32              <?php for($i = 1; $i <= 10; $i++) : ?>
33                  <option value="<?php echo $i; ?>">
34                      <?php echo $i; ?>
35                  </option>
36              <?php endfor; ?>
37              </select><br>
38
```

The last bit of code allows the user to click a link that navigates to the cart_view.php page without adding an item. The href attributes links to the index.php file and submits an action parameter of show_cart. The parameter is passed to the index.php file as part of a GET request, thus the reason the index.php file checks for both POST and GET requests in order to perform the correct action.

```
39              <label> </label>
40              <input type="submit" value="Add Item">
41          </form>
42          <p><a href=".?action=show_cart">View Cart</a></p>
43          <p>Session ID: <?php echo session_id(); ?></p>
44
45      </main>
46  </body>
47  </html>
48
```

The cart_view.php file

The first line of the PHP code checks the cart array in session; if the cart is empty the corresponding message appears for the user, otherwise, the code displays the contents of the cart in a table.

```
11      <main>
12          <h1>Your Cart</h1>
13          <?php if (empty($_SESSION['cart']) || count($_SESSION['cart']) == 0) : ?>
14              <p>There are no items in your cart.</p>
15          <?php else: ?>
```

The cart table is contained within a form that lets the user update the quantity of each item in the cart. The form uses the POST method and submits the data to the index.php file for processing. It also contains a hidden field with a name of “action” and a value of “update, which tells the index.php file to update the quantities in the cart.

```
16              <form action="." method="post">
17              <input type="hidden" name="action" value="update">
18              <table>
```

The cart table begins with a row that displays the headers for each of the columns in the table. Within this table, the <th> and <td> tags include class attributes. These attributes allow the CSS file for this application to control the alignment of the text in these columns.

The form uses a foreach loop to display each item in the cart. The loop formats the item’s cost and total as a number with two digits after the decimal. The code then displays the item’s name, cost, and total in a row. Also, it displays the quantity; however, it displays the quantity within a text field so the user can update it. The tag for the text field has a name of “newqty” with the item key in brackets, this defines the newqty field as an array. The code then defines the value of the text field as the item’s quantity.

```
19                  <tr id="cart_header">
20                      <th class="left">Item</th>
21                      <th class="right">Item Cost</th>
22                      <th class="right">Quantity</th>
23                      <th class="right">Item Total</th>
24                  </tr>
25              <?php foreach( $_SESSION['cart'] as $key => $item ) :
26                  $cost  = number_format($item['cost'],  2);
27                  $total = number_format($item['total'], 2);
28              ?>
29                  <tr>
30                      <td>
31                          <?php echo $item['name']; ?>
32                      </td>
33                      <td class="right">
34                          $<?php echo $cost; ?>
35                      </td>
36                      <td class="right">
37                          <input type="text" class="cart_qty"
38                              name="newqty[<?php echo $key; ?>]"
39                              value="<?php echo $item['qty']; ?>">
40                      </td>
41                      <td class="right">
42                          $<?php echo $total; ?>
43                      </td>
44                  </tr>
45              <?php endforeach; ?>
```

Once the item rows are generated by the foreach loop, the table displays the cart subtotal in one row and a submit button in another. It also displays instructions that explain how to update the items in the cart and how to remove items from the cart.

The Add Item link allows the user to return to the Add Item page; this is achieved by sending an action of show_add_item to the index.php page. The Empty Cart link allows the user to remove all of the items from the cart. The action of empty_cart is sent to the index.php page.

```
45                <?php endforeach; ?>
46                    <tr id="cart_footer">
47                        <td colspan="3"><b>Subtotal</b></td>
48                    <td>$<?php echo get_subtotal(); ?></td>
49                    </tr>
50                    <tr>
51                        <td colspan="4" class="right">
52                            <input type="submit" value="Update Cart">
53                        </td>
54                    </tr>
55                </table>
56                <p>Click "Update Cart" to update quantities in your
57                    cart. Enter a quantity of 0 to remove an item.
58                </p>
59                </form>
60            <?php endif; ?>
61            <p><a href=".?action=show_add_item">Add Item</a></p>
62            <p><a href=".?action=empty_cart">Empty Cart</a></p>
63            <p><a href=".?action=end_session">End Session and Delete Cookie</a></p>
64            <p>Session ID: <?php echo session_id(); ?></p>
65        </main>
66    </body>
67    </html>
```

This ends the “guided activity”; at this point, make sure the “Shopping Cart” functions properly and then upload the files to your school server. Send the link to the “Shopping Cart” using the corresponding drop box.

Continue to your next assignments:

- ***Week 12 Quiz***